Project README and Minimum Viable Product (MVP)

The first step of the project is to more formally document a written specification for the piece of software that your team will be designing and building. The below outlines the key information that should be visible in your team's readme. **As your team makes progress on your design and implementation, your team should update your readme to reflect your progress.** An up-to-date readme will make it easier for your team to prepare your final presentation at the end of the term.

> A recommended workflow is to require team members to update the readme as part of any PR that is made.

A template for your team's readme is available here: **project-readme-template-2026.md (https://q.utoronto.ca/courses/444184/files/45644698?wrap=1)** ↓ **(https://q.utoronto.ca/courses/444184/files/45644698/download?download_frd=1)**

> Use of the template is optional, but please ensure that your team's readme contains all information outlined below.
>
> If you use the template, please feel free to reorganize things as needed.

Additional guidance for what to put in your team's readme is below:

Team or Project Name

Your team can do this now or later, but at some point your team should pick a name. You can go with your group name on Quercus or something else.

Project Domain

This can just be something like "Productivity", "Education", etc...

Including a short blurb about your application is helpful here to give the reader a sense of what your project is all about.

User stories

Aim for one user story per team member. Reminder that your team can ask for feedback if you are unsure about the scope of your proposed project.

Tip: As you write your user stories, your team may also find it helpful to write a short "specification" of one to two paragraphs — similar to the todo list specification from lab. This

can help you perform the typical kind of noun-verb analysis to identify the core entities of your system. This could then go in the "Project Domain" description above.

Use cases

Based on your User Stories, identify use cases required to accomplish those user stories. Each Use Case should follow the template presented in lecture and the course notes. Decide which use cases are part of a "minimum viable product" (MVP) and which can be added later if there is time, once your program demonstrates basic functionality.

> An MVP is intended to show the viability of a product — or proof of concept — to potential clients or investors. In this case, your TAs and instructors. It might not have *everything* working, but it should show off the most important functionalities of the program and demonstrate solid design.

MVP

Each team member should "take the lead" on at least one use case in the minimum viable product. In your readme, your team will indicate who is leading each use case.
"Take the lead" means code most of:

- the Use Case Interactor and its related interfaces
- a Controller class and Presenter class that are specific to this Use Case
- the part of the UI that allows the user to trigger the Use Case
- the updating of the UI after the Use Case

"Take the lead" also means coordinate with anyone else working on code that is used as part of your use case. For example, if you are sharing a View with another Use Case, you will work with the lead on the other Use Case to ensure the View works the way you both want it to.

> You may find it useful to even start sketching some sequence diagrams for your MVP use cases.
> PlantUML is recommended, as you can save the source file in your repo and regenerate any diagrams as you update your design.

Entities for your Domain

Based on your team's chosen domain and user stories, your team should be able to pick out a few entities for your system. Depending on your domain, the number of entities will vary, but there should definitely be at least two. If your team is finding it difficult to come up with entities, you may want to think about ways to expand the scope of your project.

- For each entity, indicate its name, instance variables, and relevant methods. This can be in point form or presented as a UML class diagram as long as the information is clearly presented.

> PlantUML is again recommended, as you can save the source file in your repo and regenerate any diagrams as you update your design.

API Usage

Document any APIs your team has found that you will be able to make use of in your project.

This should include:

- A link to at least one API's documentation.
- A few bullet points or a short paragraph explaining how the API(s) will be used in the project.
- An indication of whether your team has successfully called each of the APIs listed.

> Remember that API usage is a requirement of the project, so make sure to put in some time at the start of the project to explore potential APIs to get a sense of how easy it will be to call them from your Java program. You can refer to the **API Advice (https://q.utoronto.ca/courses/444184/pages/api-advice)** and **API resources (https://q.utoronto.ca/courses/444184/pages/api-resources)** pages for ideas. Please ask on Piazza and share if your team is experimenting with a particular API or encountering issues, as others might be working in the same domain and be able to share what they have tried.

Other sections

Your team may want to include additional information, such as documentation on how to run your program or other development notes.

Final Advice

- **Start early and expect to iterate on your readme several times throughout the project.**
- Please don't be shy about sharing your draft work on Piazza, as that will benefit everyone. Instructors and your peers will be a great source of feedback as you iterate on your plans. Just toss a link to your repo on Piazza!
  - Based on user stories that teams wrote on their Week 5 worksheet in class, we put together a list of broad project directions that were mentioned in case any groups are still looking for inspiration: **Project Ideas (from Week 5 user stories shared in class) (https://q.utoronto.ca/courses/444184/pages/project-ideas-from-week-5-user-stories-shared-in-class)**
- Some APIs can be more involved to get working, so please make sure to not leave this until late in the project to get working — remember, you can write little snippets of proof-of-concept code any time.
  - Note: If your team does a good job with your design, your team will still be able to demonstrate your MVP — *even* if your API isn't fully integrated into your application — but it would be great to see it fully working!

- Beyond the above, you'll have to start thinking about design. We encourage you to start thinking about how the user interface might look for your overall program. During the second lab after Reading Week your team will be tasked with fleshing out a design for everything — including sketches of your views and plans for your use cases necessary for your MVP — but your team is welcome to get started!